

nsx-collector

사용법

NSX Edge · ESXi 에서 패킷 · 상태 · 세션 수집하기

① 설정 채우기 → ② 메뉴로 수집하기

별첨 — 커맨드 레벨 수동 수집

순서가 전부다.

앞쪽(Part 1~4)에서 설정 파일을 채우고, 뒤쪽(Part 5~6)에서 메뉴로 수집한다. 설정이 비어 있으면 메뉴에서 아무리 눌러도 수집이 되지 않는다 — 실제로 가장 많이 겪는 문제가 이것이다.

뒤쪽 별첨(A~H)에는 도구가 실제로 실행하는 명령을 전부 적었다. 도구를 올릴 수 없는 장비이거나 자동화가 말을 듣지 않을 때, 같은 수집을 손으로 할 수 있다.

수집한 결과를 해석하는 방법은 이 자료에 없다. 별도 자료로 다룬다.

도구 nsx-collector v6.3 (수집 전용)

작성일 2026-09-21

목차

Part 0	도구 소개 — 30초	3
Part 1	설정 채우기 ① — 공통 항목	5
Part 2	설정 채우기 ② — NSX Edge	7
Part 3	설정 채우기 ③ — ESXi	10
Part 4	설정 채우기 ④ — 무엇을 잡을까	12
Part 5	메뉴로 수집하기	14
Part 6	안 될 때 · 실습	17

별첨 · 커맨드 레벨 수동 수집

A	공통 — 명령을 어디서 치나	20
B	NSX Edge — 상태 수집	22
C	NSX Edge — 세션 수집	25
D	NSX Edge — 패킷 덤프	28
E	ESXi — 상태 수집	32
F	ESXi — 세션 수집 (DFW)	34
G	ESXi — 패킷 덤프	36
H	한 장 요약	39

이 자료를 읽는 법

Part 1 은 모두 읽고, 그다음은 내가 지금 작업할 장비만 읽으면 된다.

내가 작업할 장비	읽을 곳
NSX Edge	Part 1 → Part 2 → Part 4 → Part 5 · 별첨 B·C·D
ESXi 호스트	Part 1 → Part 3 → Part 4 → Part 5 · 별첨 E·F·G

도구 소개 — 30초

파일 2개가 전부다

파일	무엇
<code>nsx-collector.py</code>	수집기 본체. 이건 건드릴 일 없다
<code>nsx-collector.conf</code>	내가 채우는 것. 이 자료의 Part 1~4 가 전부 이 파일 이야기다

장비에 이 두 개만 올리면 된다. **설치할 것이 없다** — 파이썬 표준 라이브러리만 쓴다.

장비를 알아서 구분한다

같은 파일인데 **NSX Edge**에서는 **Edge 메뉴가**, **ESXi**에서는 **ESXi 메뉴가** 나온다. 잘못된 명령을 고를 일이 없다.

무엇을 모으나

장비	수집 항목
NSX Edge	· LB T1 서비스 인터페이스 패킷 캡처 · VPC T1 업링크 패킷 캡처 · T0 업링크 패킷 캡처 · 인터페이스 · CPU · 메모리 카운터 · 방화벽 연결 테이블, LB 상태 · 풀
ESXi	· DFW 적용 전(pre) 패킷 캡처 · DFW 적용 후(post) 패킷 캡처 · DFW 플로우 테이블 · 룰 · pass/drop 카운터 · 스위치포트 · 업링크 NIC 카운터

왜 이걸 쓰나 — 한 줄로

세션 테이블 같은 것은 지나가면 영영 못 얻는다. 증상이 있는 그 순간에 무엇을 떠야 하는지를 미리 정해 두고 한 번에 도는 것, 그게 전부다.

올리고 실행하기

```
scp nsx-collector.py nsx-collector.conf root@<장비>:/root/nsxc/
```

```
ssh root@<장비>
```

```
cd /root/nsxc
```

```
python3 nsx-collector.py # 메뉴가 뜬다
```

NSX Edge 는 반드시 root 로

`admin` 으로 들어가면 NSX CLI 셸이라 `python3` 가 실행되지 않는다.

설정 채우기 ①

공통 항목

Edge · ESXi 어느 쪽이든 똑같이 해당되는 항목.

먼저 알아야 할 규칙 3가지

- 규칙 1 — 한 파일로 두 장비를 다 쓴다**

이 파일 하나에 Edge 용 칸과 ESXi 용 칸이 **같이 들어 있다**. 지금 장비가 필요로 하는 것만 채우고 **나머지는 그냥 비워 둔다**.
- 규칙 2 — 빈 값은 "안 함"이다**

비워 두면 **"그건 수집하지 않는다"**는 뜻이다. "조건이 없으니 전부"가 아니다.
딱 하나 예외가 캡처 필터다 — 그건 Part 4 에서 따로 다룬다.
- 규칙 3 — 윈도우 편집기로 열지 말 것**

메신저로 붙여넣거나 윈도우 메모장으로 저장하면 **줄바꿈이 바뀌어 실행을 거부한다**. 장비에서 `vi` 로 고치는 게 가장 안전하다.

공통 항목 — 5개뿐

키	무엇	안 채우면
CASE_ID 선택	케이스 번호. 화면에만 표시된다	표시 안 함
TAG 선택	결과 파일 이름에 들어갈 짧은 이름	호스트명을 쓴다
OUT 선택	결과가 쌓일 곳	Edge <code>/var/dump/nsx-collect</code> ESXi <code>/tmp/nsx-collect</code> ← 위험
MIN_FREE_MB 선택	이 용량 아래로 떨어지면 자동 정지	Edge 1024 / ESXi 64

ESXi 라면 OUT 을 꼭 바꾼다

ESXi 의 `/tmp` 는 약 **250MB** 짜리 메모리 디스크다. 캡처를 조금만 돌려도 꼭 찬다.

```
OUT="/vmfs/volumes/<데이터스토어>/nsxc"
```

`df -h` 로 여유부터 본다. 데이터스토어 이름에 **공백이나 괄호가 든 것은 피한다** (예: `datastore1 (1)`).

얼마나 오래 · 얼마나 크게

키	기본	설명
패킷 캡처에만 적용		
<code>CAP_SECS</code>	600	몇 초 동안 캡처할지
<code>CAP_FILESIZE</code>	100	파일 하나당 MB
<code>CAP_FILECOUNT</code>	5	파일 몇 개까지. 넘으면 오래된 것부터 덮어쓴다
<code>CAP_SNAPLEN</code>	전체	패킷당 저장할 바이트. 헤더만 보면 되면 <code>"128"</code> 등으로 줄인다
상태 · 세션 수집에만 적용		
<code>INTERVAL</code>	30	몇 초마다 한 번씩 뜯지
<code>DURATION</code>	1200	총 몇 초 동안 뜯지

용량 계산은 곱셈 하나면 된다

`CAP_FILESIZE` × `CAP_FILECOUNT` = 캡처 한 지점이 쓰는 최대 용량

기본값이면 $100 \times 5 = 500\text{MB}$. 캡처 지점이 2곳이면 1GB. 남은 디스크와 비교해 보고 정한다.

처음 연습할 때는 작게

```
CAP_SECS=30
CAP_FILESIZE=10
CAP_FILECOUNT=2      # 최대 20MB 로 끝난다
```

설정 채우기 ②

NSX Edge

Edge 에서 작업할 때만 읽으면 된다.

필수는 딱 하나 — 어디서 캡처할지

아래 둘 중 **최소 하나**만 채우면 된다. 둘 다 채워도 된다.

키	무엇
LIF_LBT1_SVC 필수	LB T1 의 서비스 인터페이스 UUID
LIF_VPCT1_UPLINK 필수	VPC T1 의 업링크 인터페이스 UUID

이게 비어 있으면 캡처가 아예 시작을 안 한다

메뉴에서 **start** 를 눌러도 **캡처 항목이 idle** 로만 표시되고 **아무 일도 일어나지 않는다**. 카운터와 세션은 잘 도는데 캡처만 안 되는 모습이라 "패킷 덤프가 안 된다"로 보인다.

실제로 가장 많이 겪는 문제다. Part 6 에 실제 사례가 있다.

찾는 방법 — 명령 2개

```
su admin -c "get logical-routers"
# 라우터 목록이 나온다. LB 용 T1 과 VPC 용 T1 의 UUID 를 확인

su admin -c "get logical-router <위에서 찾은 UUID> interfaces"
# 인터페이스 목록에서 Port-type 을 본다

Port-type: service    → 이 UUID 를 LIF_LBT1_SVC     에
Port-type: uplink     → 이 UUID 를 LIF_VPCT1_UPLINK  에
```

둘 다 조회 명령이다. 장비 설정을 바꾸지 않는다.

Active 인 Edge 에서 해야 한다

T1 은 보통 Edge 2대가 **Active / Standby** 로 나눠 갖는다. **Standby** 쪽에서 캡처하면 패킷이 0건이다. 트래픽이 그쪽으로 안 가기 때문이다.

어느 쪽이 Active 인지는 **메뉴의 check** 가 알려 준다(Part 5). 캡처 전에 반드시 확인한다.

T0 는 비워 두는 게 맞다

키	설명
LIF_T0_UPLINK	비워 둔다. 비워 두면 도구가 이 장비의 T0 를 알아서 찾아 업링크 전부에서 캡처한다(보통 2개)
CAPTURE_T0	"yes" 로 하면 start 에 T0 캡처도 포함된다. 기본은 제외 — T0 는 사이트로 들어오는 트래픽 전부라 너무 시끄럽다. 필요하면 메뉴 대신 cap-t0 로 한 번만 돌리면 된다

선택 — 채우면 더 정확해지는 것들

안 채워도 캡처는 된다. 채우면 필터가 자동으로 만들어져 쓸데없는 패킷이 줄어든다.

키	무엇
VIP	LB 가상 서버 IP
NAT_IP	DNAT 되기 전의 공인 주소 — 아래 설명 참조
LB_SNAT_IP	LB 가 백엔드로 나갈 때 쓰는 주소
CLIENT_IP	클라이언트 주소 또는 대역
SVC_PORT	클라이언트 → VIP 포트 (예: 443, 1812)
NODE_PORT	LB → 백엔드 포트 (예: 80, 31820)

NAT_IP 를 빠뜨리면 VPC T1 캡처가 0건이 된다

VPC T1 업링크는 NAT 되기 전 구간이다. 그래서 그곳에는 VIP 주소가 아예 나타나지 않는다. VIP 로만 필터를 걸어 두면 당연히 아무것도 안 잡힌다.

랩 실측: NAT_IP 를 넣기 전 0건 → 넣은 뒤 370건.

선택 — 상태도 같이 볼 라우터

키	설명
T1_LB_SR_UUID T1_VPC_SR_UUID T0_SR_UUID	여기 적은 라우터의 카운터 · HA 상태를 같이 수집한다. 라우터 하나당 한 번 뜰 때마다 약 2.5초 가 걸리니 필요한 것만 적는다
LB_UUID · LB_POOL_UUIDS	LB 상태 · 풀 · 가상서버를 같이 수집

완성 예시 — Edge

```
# --- 공통 (Part 1) ---
CASE_ID="ISCPSR-12345"
CAP_SECS=300

# --- 필수: 어디서 캡처할지 ---
LIF_LBT1_SVC="11111111-2222-3333-4444-555555555555"
LIF_VPCT1_UPLINK="66666666-7777-8888-9999-aaaaaaaaaaaa"

# --- 선택: 필터를 정확하게 ---
VIP="10.1.1.10"
NAT_IP="203.0.113.10"           # ← 빠뜨리면 VPC T1 이 0건
LB_SNAT_IP="10.1.1.20"
SVC_PORT="443"
NODE_PORT="80"

# --- 선택: 상태도 같이 ---
T1_LB_SR_UUID="bbbbbbbb-cccc-dddd-eeee-ffffffffffff"
T1_VPC_SR_UUID="12345678-90ab-cdef-1234-567890abcdef"
```

설정 채우기 ③

ESXi

ESXi 호스트에서 작업할 때만 읽으면 된다. Edge 보다 훨씬 단순하다.

필수는 딱 하나 — 어느 VM 을 볼지

키	무엇
WORKER_VMS 필수	대상 VM 이름. 여러 개면 공백으로 구분

찾는 방법 — 명령 1개

```
vim-cmd vmsvc/getallvms
# 여기 나오는 이름을 그대로 적는다

WORKER_VMS="web01 web02"
```

이 호스트에 없는 VM 을 적어도 된다

없는 VM 은 "이 호스트에 없다"고 알려 주고 건너뛴다. 그래서 여러 호스트에 같은 설정 파일을 뿌려도 된다 — 각자 자기 호스트에 있는 VM 만 수집한다.

이름에 공백이 든 VM 은 못 쓴다

공백으로 구분하는 목록이라 담을 수 없다. 도구가 목록에서 빼고 이유를 알려 준다. 괄호가 든 이름 (SupervisorControlPlaneVM_(2))은 정상 처리된다.

나머지 2개 — 선택

키	설명
WORKER_VNIC 선택	비워 두기를 권장. 비우면 그 VM 의 모든 vNIC 을 본다. 특정 vNIC 만 보고 싶을 때만 적는다
UPLINK_NICS 선택	물리 NIC 카운터를 같이 볼 때. "vmnic0 vmnic1"

완성 예시 — ESXi

```
# --- 공통 (Part 1) ---
CASE_ID="ISCPSR-12345"
TAG="esxi02"
OUT="/vmfs/volumes/ds02/nsxc"      # ← /tmp 쓰지 말 것
CAP_SECS=300

# --- 필수: 어느 VM ---
WORKER_VMS="web01 web02"

# --- 선택 ---
WORKER_VNIC=""                      # 비움 = 모든 vNIC
UPLINK_NICS="vmnic0 vmnic1"

# --- 무엇을 잡을지 (Part 4) ---
FILTER="tcp port 80"
```

ESXi 는 설정이 이게 전부다

Edge 처럼 UUID 를 찾아 넣을 일이 없다. **VM 이름만 정확하면 된다.**

설정 채우기 ④

무엇을 잡을까

캡처 대상을 좁히는 방법. Edge · ESXi 공통.

방법은 두 가지, 하나만 고르면 된다

방법 1 — FILTER 에 직접 쓴다 (권장)

tcpdump 표현식을 그대로 쓴다. **이걸 쓰면 다른 모든 설정보다 우선한다.**

```
FILTER="tcp port 80"
FILTER="host 10.1.1.10 and (udp port 1812 or udp port 1813)"
FILTER="(host 10.1.1.10 or host 10.1.1.11) and not tcp port 22"
FILTER="net 10.1.1.0/28 and udp"
```

방법 2 — 칸을 채운다 (FILTER 를 비웠을 때만)

키	무엇	예
HOSTS	주소	"10.1.1.10 10.1.1.0/28"
PROTO	프로토콜	"udp" · "udp tcp"
PORTS	포트	"1812 1813"

Edge 라면 Part 2 의 `VIP` · `NAT_IP` · `SVC_PORT` 등도 여기에 함께 쓰인다. 도구가 캡처 지점마다 알맞은 필터를 따로 만들어 준다.

실제로 만들어지는 필터 (Part 2 예시 설정 기준)

```
LB T1    ((host 10.1.1.10) and (tcp and port 443))
         or ((host 10.1.1.20) and (tcp and port 80))
         or (icmp and (host 10.1.1.10 or host 10.1.1.20))

VPC T1   (host 10.1.1.10 or host 203.0.113.10) and ((tcp and port 443) or icmp)
```

지점마다 봐야 할 주소가 다르다는 것을 도구가 알아서 처리한다 — LB T1 은 VIP 와 SNAT 주소를, VPC T1 은 NAT 전 주소를 본다.

꼭 기억할 합정 2가지

① and 와 or 는 우선순위가 같다 — 괄호를 쓴다

수학처럼 `and` 가 먼저 계산되지 **않는다**. 왼쪽부터 순서대로 읽는다.

내가 의도한 것 `a or (b and c)`
실제로 해석되는 것 `(a or b) and c` ← 다르다!

해결: 괄호를 직접 쓴다

② 전부 비우면 모든 패킷을 잡는다

`FILTER` 도 비우고 칸도 다 비우면 그 인터페이스의 패킷을 전부 잡는다. 디스크가 순식간에 찬다. 의도한 게 아니면 뭐라도 채운다.

넣고 나서 반드시 확인

메뉴에서 `config` 를 누르면 실제로 만들어진 필터식을 보여 준다. 문법이 틀리면 시작 전에 걸러 준다. 눈으로 한 번 보는 습관을 들인다.

메뉴로 수집하기

설정이 끝났으면 이제 메뉴만 순서대로 누르면 된다.

메뉴 항목

항목	하는 일
config	내 설정값과 만들어진 필터식을 보여 준다
discover	장비에 물어서 설정을 대신 채워 준다 (다음 절)
selftest	돌리기 전에 맞아야 할 것 전부 점검
check	Edge: 어느 노드가 Active 인지 ESXi: VM 목록 · 남은 찌꺼기 · 디스크
rehearse	캡처 빼고 한 번만 떠 본다 (연습)
start	수집 시작 (백그라운드)
status · watch	진행 상황
stop	멈춘다 — 파일은 남긴다
wipe	멈추고 우리가 만든 것만 지운다

discover — 설정을 대신 채워 주는 기능

UUID 를 직접 찾기 번거로우면 이걸 쓴다. 장비에 물어보고 목록을 보여 준 다음, 고르면 설정에 써 준다.

장비	물는 순서
Edge	① 로드밸런서 고르기 → ② 가상 서버 고르기 → ③ LB 앞단 T1 고르기 → ④ T0 는 자동 → ⑤ 공인 주소 (NAT_IP)만 직접 입력
ESXi	① DFW 필터가 있는 VM 중에서 고르기 → ② 업링크 NIC 확인

마지막에 바뀔 항목만 보여 주고 y 를 눌러야 저장한다.

```
* WORKER_VMS      web01 web02
* UPLINK_NICS      vmnic0 vmnic1

write this into nsx-collector.conf? [y/N]
```

discover 는 사람이 앉아서 눌러야 한다

ssh 장비 '명령' 처럼 한 줄로 원격 실행하면 아무것도 저장되지 않는다. 장비에 접속해서 메뉴로 실행하거나, Part 2 · 3 을 보고 직접 채운다.

수집 순서 — 이 순서대로만 하면 된다

순서	명령	확인할 것
1	selftest	ALL GOOD 이 나와야 한다. 아니면 진행하지 않는다
2	check	Edge 면 Active – capture HERE 가 보이는지. 남은 찌꺼기가 없는지, 디스크 여유가 있는지
3	rehearse	캡처 없이 한 번 떠 본다. 파일이 실제로 만들어지는지 확인
4	start	증상이 나타나는 시점에 시작
5	status	캡처가 running 인지. idle 이면 설정이 비어 있다
6	stop	수집 종료. 파일은 남는다
7	결과 복사	scp -r root@<장비>:<OUT>/run-*/ ./
8	wipe	철수. 우리가 만든 것만 지운다

고객 장비에서는 --dry-run 부터

```
python3 nsx-collector.py stop --dry-run
python3 nsx-collector.py wipe --dry-run
```

무엇을 멈추고 무엇을 지울지 보여 주기만 하고 아무것도 바꾸지 않는다. 확인한 다음에 진짜로 실행한다.

selftest 에서 자주 걸리는 줄

이렇게 나오면	이렇게 한다
target VMs on this host – 0 vNIC(s)	WORKER_VMS 가 비었거나 이름이 틀렸다 → Part 3
capture interface set – no	LIF_LBT1_SVC / LIF_VPCT1_UPLINK 가 비었다 → Part 2
free space 가 바닥	OUT 을 데이터스토어로 바꾼다 → Part 1
filter expression 오류	괄호를 확인한다 → Part 4

메뉴 없이 하나만 돌리기

Edge :	python3 nsx-collector.py cap-lbt1 30	# LB T1 캡처 30초
	python3 nsx-collector.py cap-vpct1 30	# VPC T1 캡처 30초
	python3 nsx-collector.py cap-t0 30	# T0 캡처 30초
	python3 nsx-collector.py stats-run	
	python3 nsx-collector.py sess-run	
ESXi :	python3 nsx-collector.py cap-pre 30	# DFW 적용 전
	python3 nsx-collector.py cap-post 30	# DFW 적용 후
	python3 nsx-collector.py stats-run	
	python3 nsx-collector.py dfw-run	

안 될 때 · 실습

캡처가 안 될 때 — 이 순서로 본다

순서	확인	증상 · 해결
1	설정에 캡처 지점이 있나	<code>status</code> 에서 캡처가 <code>idle</code> , span 인터페이스가 <code>none</code> → 아예 시작을 안 한 것 . Part 2 의 <code>LIF_*</code> 를 채운다
2	Active 인 Edge 인가	Standby 에서는 0건 . <code>check</code> 로 확인
3	NAT_IP 를 넣었나	VPC T1 캡처가 0건이면 심중팔구 이것. Part 2
4	필터가 너무 좁은가	<code>config</code> 로 만들어진 식을 눈으로 본다. Part 4
5	정말 트래픽이 흐르나	직접 부하를 만들어 본다

실제 사례 — 2026-09-21

Edge 에서 수집을 돌렸는데 카운터와 세션은 도는데 캡처만 `idle` 이었다.

```
LB T1 capture      [-----]      idle
VPC T1 capture     [-----]      idle
counters           [-----]      3%  running  19m22s left
sessions           [-----]      3%  running  19m23s left

SPAN INTERFACES
none
```

원인: 설정 파일의 `LIF_LBT1_SVC` 와 `LIF_VPCT1_UPLINK` 가 **둘 다 비어 있었다**. `VIP` 와 `NAT_IP` 도 `<VIP>` , `<NAT_IP>` 자리표시자 그대로였다.

해결: 값을 채운 설정으로 다시 돌리니 **30초에 14,455 패킷**이 잡혔다.

```
04:11:31 capture lbt1-svc
  interface : 11111111-...  (span-0)
  duration  : 30s
14455 packets captured
0 packets dropped by kernel
```

카운터·세션이 잘 도는 것에 속으면 안 된다. 그것들은 캡처 지점이 필요 없어서 설정이 비어 있어도 잘 돈다.

실습

#	할 것	확인할 것
1	메뉴를 연다	Edge 와 ESXi 에서 항목이 다르게 나오는 것
2	<code>config</code> 를 본다	내 설정에서 어떤 필터식이 만들어졌는지
3	설정을 직접 고친다	ESXi 에서 <code>WORKER_VMS</code> 를 2개 → 1개로 줄이고 <code>selftest</code> 를 다시 → <code>vNIC</code> 개수가 바뀐다
4	일부러 비워 본다	Edge 에서 <code>LIF_*</code> 를 비우고 <code>selftest</code> → <code>capture interface set - no</code> 가 나오는 것
5	<code>check</code>	Edge 에서 어느 노드가 Active 인지
6	<code>rehearse</code>	캡처 없이 파일이 만들어지는 것
7	<code>cap-lbt1 30</code> + 부하	패킷이 실제로 잡히는 것
8	<code>wipe --dry-run</code> → <code>wipe</code>	지울 목록을 먼저 보는 습관

실습 7번에서 쓸 부하 생성 명령이다.

```
seq 1 2000 | xargs -P 20 -I{} curl -sk -o /dev/null -m 5 https://<서비스주소>/
```

오늘 꼭 가져갈 것 3가지

1. **설정이 먼저다.** 캡처 지점(`LIF_*` 또는 `WORKER_VMS`)이 비어 있으면 메뉴를 아무리 눌러도 안 된다
2. **`selftest` 와 `check` 없이 시작하지 않는다.** Standby 에서 캡처하면 0건이다
3. **고객 장비에서는 `--dry-run` 먼저**

별첨

커맨드 레벨 수동 수집

도구가 실제로 실행하는 명령 전수 — 상태 · 세션 · 패킷 덤프.

언제 쓰나

- 도구를 올릴 수 없는 장비 — 반입이 안 되거나 승인이 늦을 때
- 자동화가 말을 듣지 않을 때 — selftest 는 통과했는데 결과가 이상할 때, 손으로 같은 것을 떼서 대조한다
- 딱 하나만 급히 봐야 할 때 — 세션 테이블 한 장, 카운터 한 번
- 도구가 무엇을 하는지 이해할 때

여기 적힌 명령의 출처

전부 `nsx-collector.py` 소스에서 그대로 뽑았다. 도구가 내부에서 이 명령들을 돌리고 출력을 정해진 이름의 파일로 떨군다. 따라서 손으로 한 결과와 도구가 만든 결과는 같다.

NSX 4.2.4 Edge · ESXi 8.0.3 에서 실측 확인된 것이며, 버전에 따라 없는 명령이 있다 — 그런 항목은 본문에 따로 표시했다.

표기 규칙

표기	뜻
<LIF-UUID>	논리 인터페이스 UUID — 설정의 <code>LIF_LBT1_SVC</code> 등
<SR-UUID>	서비스 라우터 UUID — <code>T1_LB_SR_UUID</code> 등
<filter>	DFW 필터 이름 (<code>nic-12345-eth0-vmware-sfw.2</code> 형태)
<VM>	VM 이름

공통 — 명령을 어디서 치나

NSX Edge — 두 가지 셸이 있다

Edge 에는 **리눅스 셸**과 **NSX CLI** 두 가지가 있고, 명령이 갈린다.

셸	어떻게 들어가나	여기서 치는 것
리눅스 셸	<code>ssh root@<edge></code>	<code>tcpdump</code> , <code>ip link</code> , <code>timeout</code>
NSX CLI	<code>ssh admin@<edge></code>	<code>get ...</code> , <code>set capture session ...</code>

root 로 들어가서 NSX CLI 명령을 치는 방법

도구는 root 셸에서 `su admin -c` 로 NSX CLI 명령을 던진다. 이 방식이면 셸을 오가지 않아도 되고, 스크립트로 묶기도 쉽다.

```
su admin -c "get interfaces"
su admin -c "get logical-routers"
```

이 별첨의 Edge 명령은 전부 이 형태로 적는다. admin 셸에 직접 들어가 있다면 `su admin -c "..."` 를 빼고 **안쪽 명령만** 치면 된다.

admin 셸에서는 리눅스 명령이 안 된다

`admin` 으로 로그인하면 NSX CLI 셸이라 `tcpdump`, `python3`, `ls` 같은 것이 `% Command not found` 로 거부된다. 패킷 덤프는 반드시 **root 셸**에서 해야 한다.

ESXi — 셸 하나뿐

```
ssh root@<esxi>
```

모든 명령을 여기서 친다. 다만 ESXi 의 `timeout` 은 문법이 일반 리눅스와 다르다 — D · G 장에서 다룬다.

결과를 어디에 남길 것인가

도구는 `run-<호스트>-<날짜>-<시각>/` 아래 `state/` · `session/` · `pcap/` 세 폴더로 나눠 담는다. 손으로 할 때도 같은 구조를 만들어 두면 나중에 분석기로 읽을 수 있다.

```
D=/var/dump/manual-$(hostname)-$(date +%Y%m%d-%H%M%S) # Edge
D=/vmfs/volumes/<데이터스토어>/manual-$(date +%Y%m%d-%H%M%S) # ESXi
mkdir -p $D/state $D/session $D/pcap
```

ESXi에서는 /tmp 를 쓰지 말 것

약 250MB 메모리 디스크라 캡처가 금방 멈춘다. 반드시 데이터스토어로.

NSX Edge — 상태 수집

도구의 `stats-run` 이 돌리는 명령 전부. 카운터와 HA 상태.

B-1 → `state/30-edge-interfaces.txt`

인터페이스 목록과 카운터

```
su admin -c "get interfaces"
```

Edge 의 모든 인터페이스와 RX/TX 를 한 번에 본다. 가장 먼저 보는 것.

B-2 → `state/32-edge-dataplane-cpu.txt`

데이터플레인 CPU

```
su admin -c "get dataplane cpu stats"
```

패킷 처리 코어가 포화됐는지. 드롭이 여기서 나는지 확인할 때.

B-3 → `state/33-edge-port-<포트>-stats.txt`

물리(fastpath) 포트별 카운터

```
su admin -c "get interface fp-eth0"
su admin -c "get interface fp-eth1"
```

실측 함정 — 명령 이름이 다르다

NSX 4.2.4 에는 `get interfaces stats` 라는 명령이 아예 없다 (% Command not found). `get interface <이름>` 이 맞고, 이 출력에 이미 RX/TX 패킷·바이트·에러·드롭이 다 들어 있다.

B-4 → `state/34-edge-router-<앞8자>-interface-stats.txt`

라우터별 인터페이스 통계

```
su admin -c "get logical-router <SR-UUID> interfaces stats"
```

설정의 `T0_SR_UUID` · `T1_VPC_SR_UUID` · `T1_LB_SR_UUID` 에 적은 라우터마다 한 번씩. 라우터 하나당 약 2.5초가 걸리니 필요한 것만.

B-5 → state/35-edge-router-<앞8자>-ha-state.txt

라우터 HA 상태 ★

```
su admin -c "get logical-router <SR-UUID> high-availability status"
```

읽는 방향을 반드시 확인할 것

출력 맨 위 **state** 줄이 지금 접속한 이 노드다. 아래쪽 **Peer Routers** 블록은 상대방 이야기다.

이걸 뒤집어 읽어 Standby 에서 캡처하는 사고가 실제로 난다. Standby 에서 뜨면 패킷이 0건이다.

B-6 → state/36-edge-cpu.txt · 37-edge-memory.txt

CPU · 메모리

```
su admin -c "get cpu-stats"
su admin -c "get memory"
```

실측 함정

`get system-stats` 는 NSX 4.2.4 에 없다. 위 두 개로 나눠서 봐야 한다.

B-7. 반복해서 쓰려면

도구는 **INTERVAL** 초마다 **DURATION** 초 동안 반복한다. 손으로 하면 이렇게 된다.

```
D=/var/dump/manual-$(hostname)-$(date +%Y%m%d-%H%M%S); mkdir -p $D/state

END=$(( $(date +%s) + 1200 ))          # DURATION 1200초
while [ $(date +%s) -lt $END ]; do
    TS=$(date +%H%M%S)
    su admin -c "get interfaces"         > $D/state/30-interfaces-$TS.txt
    su admin -c "get dataplane cpu stats" > $D/state/32-dpcpu-$TS.txt
    su admin -c "get interface fp-eth0"  > $D/state/33-fp-eth0-$TS.txt
    su admin -c "get interface fp-eth1"  > $D/state/33-fp-eth1-$TS.txt
    sleep 30                             # INTERVAL 30초
done
```

왜 반복해서 뜨나

카운터는 **누적값**이라 한 번 떠서는 아무 의미가 없다. **두 시점의 차이**가 그 사이에 무슨 일이 있었는지 말해 준다. 최소 2회, 되도록 여러 번 뜬다.

NSX Edge — 세션 수집

도구의 `sess-run` . 방화벽 연결 테이블과 LB 상태.

이 장이 가장 중요하다

방화벽 연결 테이블은 **지나가면 영영 못 얻는다**. 증상이 있는 그 순간에 쓰지 않으면 사후에는 방법이 없다.

C-1 → session/50-edge-fw-conn-count-...txt

방화벽 연결 개수

```
su admin -c "get firewall <LIF-UUID> connection count"
```

먼저 개수만 본다. 테이블이 클 때 전체를 쓰기 전에 규모를 가늠한다.

C-2 → session/51-edge-fw-conn-table-...txt

방화벽 연결 테이블 전체 ★

```
su admin -c "get firewall <LIF-UUID> connection"
```

출력은 이런 모양이다.

```
0x...: 10.1.1.20:54982 -> 10.2.2.30:80 (10.1.1.10:80)
      ↑ 괄호 안에 NAT 매핑
```

괄호 안에 NAT 이전 주소가 같이 나온다. LB 가 경로에 있을 때 "이 연결이 원래 어디로 온 것인지"를 이것 하나로 알 수 있다.

실측 함정 — SR UUID 를 넣으면 안 된다

이 명령은 **논리 인터페이스(LIF) UUID** 를 받는다. 서비스 라우터(SR) UUID 를 넣으면 `% Invalid value for argument <uuid>` 가 나오고, **파일에는 데이터 대신 에러가 담긴다**.

도구 v5 가 여기서 아무것도 못 모았던 원인이 이것이다. NSX 4.2.4 실측.

설정에 적은 LIF **전부**에 대해 돌린다 — `LIF_LBT1_SVC` , `LIF_VPCT1_UPLINK` , `LIF_T0_UPLINK` , `LIF_VPCT1_DOWNLINK` .

C-3 → session/52-edge-lb-...txt

로드밸런서 상태와 통계

```
su admin -c "get load-balancer <LB-UUID> status"
su admin -c "get load-balancer <LB-UUID> stats"
```

C-4 → session/54-edge-lb-virtualservers-...txt

가상 서버 목록

```
su admin -c "get load-balancer <LB-UUID> virtual-servers"
```

C-5 → session/55-edge-lb-pool-<앞8자>-status.txt

풀 상태 — 멤버가 살아 있나

```
su admin -c "get load-balancer <LB-UUID> pool <POOL-UUID> status"
```

백엔드가 전부 DOWN 인지 여기서 확인한다. 풀이 여러 개면 각각.

C-6. UUID 를 모를 때 — 찾는 명령

명령	얻는 것
su admin -c "get logical-routers"	SR UUID 목록
su admin -c "get logical-router <SR-UUID> interfaces"	LIF UUID — Port-type 이 service / uplink
su admin -c "get load-balancers"	LB UUID 목록
su admin -c "get load-balancer <LB-UUID> virtual-servers"	VIP · 포트 · 풀
su admin -c "get load-balancer <LB-UUID> pool <POOL> snat-pools"	SNAT 주소

C-7. 반복해서 뜨려면

```
D=/var/dump/manual-$(hostname)-$(date +%Y%m%d-%H%M%S); mkdir -p $D/session
LIF=<LIF-UUID>

END=$(( $(date +%s) + 1200 ))
while [ $(date +%s) -lt $END ]; do
    TS=$(date +%H%M%S)
    su admin -c "get firewall $LIF connection count" > $D/session/50-conn-count-$TS.txt
    su admin -c "get firewall $LIF connection" > $D/session/51-conn-table-$TS.txt
    sleep 30
done
```

세션 테이블도 여러 번 떼야 한다

연결이 **생겼다 사라지는 과정**을 봐야 하기 때문이다. 한 장만 있으면 "그때 세션이 있었나 없었나"만 알 수 있고, **언제 사라졌는지는 모른다**.

NSX Edge — 패킷 덤프

도구의 `cap-lbt1` · `cap-vpct1` · `cap-t0` . span 미러를 만들고 tcpdump 로 받는다.

원리 — 인터페이스를 직접 뜯 수 없다

NSX Edge 의 논리 인터페이스는 **tcpdump** 로 직접 지정할 수 없다. 그래서 **span 미러**를 만들어 `span-N` 이라는 가상 인터페이스를 띄우고, 거기에 tcpdump 를 붙인다.

```
<LIF-UUID> → (미러) → span-0 → tcpdump
```

3단계로 한다

D-1 · NSX CLI

① span 미러 생성

```
su admin -c "set capture session 0 interface <LIF-UUID> direction dual"
```

`0` 은 세션 번호(0~3). `dual` 은 양방향.

생성되면 `span-0` 인터페이스가 리눅스 쪽에 나타난다. 확인:

```
ip link show span-0
```

최대 10초쯤 기다려야 할 수 있다

도구도 1초 간격으로 10번까지 확인하고, 안 뜨면 **세션을 지우고 실패** 처리한다 (미러를 남기지 않기 위해). 안 뜨면 **LIF UUID** 를 다시 확인한다.

D-2 · 리눅스 셸 (root)

② tcpdump 로 받기 ★

```
timeout -s INT 300 tcpdump -nei span-0 \  
-Z root \  
-C 100 -W 5 \  
-w /var/dump/manual/pcap/10-lbt1svc-$(date +%H%M%S).pcap \  
'<filter>'
```

옵션	왜 필요한가
<code>-Z root</code>	반드시 넣는다. tcpdump 는 기본적으로 권한을 낮추는데, 그러면 링 버퍼의 다음 파일을 못 만든다
<code>timeout -s INT 300</code>	300초 뒤 INT 시그널 로 정상 종료. 그래야 마지막 파일이 온전하다
<code>-C 100 -W 5</code>	파일당 100MB, 5개까지 = 최대 500MB . 넘으면 오래된 것부터 덮어쓴다
<code>-nei</code>	n 이름해석 안 함 · e L2 헤더 표시 · i 인터페이스
<code>-s 128</code>	(선택) 패킷당 128바이트만. 헤더만 볼 때 용량 절약

D-3 · NSX CLI

③ span 미러 삭제 — 반드시

```
su admin -c "del capture session 0"
```

남겨두면 안 된다

미러는 **계속 패킷을 복사**한다. 운영 장비에 남기면 부하가 된다. 도구는 tcpdump 가 어떻게 끝나든 **finally** 로 반드시 지운다. 손으로 할 때는 **잊기 쉬우니 먼저 메모해 둘 것**.

남은 미러 확인:

```
su admin -c "get capture session 0"  
ip link show | grep span
```

D-4. 한 번에 붙여 쓰기

```
LIF=<LIF-UUID>
D=/var/dump/manual-$(date +%Y%m%d-%H%M%S); mkdir -p $D/pcap

su admin -c "set capture session 0 interface $LIF direction dual"
sleep 3; ip link show span-0 || echo "span 이 안 떴다 - LIF 확인"

timeout -s INT 300 tcpdump -nei span-0 -Z root -C 100 -W 5 \
-w $D/pcap/10-lbt1svc-$(date +%H%M%S).pcap \
'host 10.1.1.10 and tcp port 443'

su admin -c "del capture session 0"      # ← 잊지 말 것
ls -la $D/pcap/
```

D-5. 어느 LIF 를 뜯 것인가

지점	무엇이 보이냐	세션 번호 예
LB T1 서비스	VIP 로 들어온 트래픽, LB → 백엔드	0
VPC T1 업링크	NAT 이전 트래픽 — VIP 가 안 보인다	1
T0 업링크	사이트로 들어오는 전부. 매우 시끄럽다	2, 3

캡처가 0건일 때 — 순서대로 본다

1. 이 Edge 가 Standby 인가 — B-5 의 HA 상태 확인. Standby 면 0건이다
2. 그 시간 동안 매칭되는 트래픽이 실제로 있었나
3. 필터가 이 지점에 안 맞나 — VPC T1 업링크는 NAT 이전이라 VIP 가 절대 안 나타난다. 공인/NAT 주소로 걸거나 필터를 비운다

이 세 지점은 one-arm LB 기준이다 — inline 이면 다르다

도구의 Edge 캡처 지점은 **one-arm LB**(LB 가 T1 의 **서비스 인터페이스**에 붙고 SNAT 이 필수인 구조)를 전제로 만들어졌다.

지점	one-arm	inline
T0 업링크	○	○ 그대로
T1 업링크 (클라이언트 쪽)	○	○ 그 T1 업링크를 <code>LIF_VPCT1_UPLINK</code> 에
LB 서비스 IF	○	해당 없음 — 서비스 IF 가 없다
T1 다운링크 (백엔드 쪽)	해당 없음	캡처 불가 — 도구의 캡처 지점에 없다

inline에서는 다운링크 공백을 **ESXi** 쪽으로 메운다 — 백엔드 VM 의 DFW `pre / post` 가 "LB 가 보낸 패킷이 서버에 도착하는 지점"을 보여 준다 (부록 G). 결국 **T0 업링크 · T1 업링크 · ESXi pre · ESXi post** 네 지점이면 구간 판정이 된다.

또한 inline 은 **SNAT 이 선택**이라 `LB_SNAT_IP` 가 비기 쉽다. 그러면 단순 필드로 만든 필터에서 **LB→백엔드 구간 절이 통째로 빠진다**. inline 이면 `FILTER` 를 직접 쓰는 편이 안전하다.

```
FILTER="(host <VIP> or host <백엔드IP>) and tcp port <포트>"
```

`LIF_VPCT1_DOWNLINK` 는 캡처는 안 되지만 **방화벽 연결 테이블은 수집된다**(부록 C). inline 이면 이 값은 꼭 채운다.

T0 는 A/A — 모든 Edge 에서 동시에 떼야 한다

T0 는 보통 Active/Active 라 트래픽이 노드마다 나뉜다. **한 노드만 뜨면 절반만 보인다**. 그리고 T0 업링크는 보통 **2개**라 노드당 2개를 떼야 한다.

ESXi — 상태 수집

도구의 `stats-run` (ESXi). 스위치포트와 업링크 카운터.

E-1 → state/40-esxi-switchport-list.txt

스위치포트 목록 ★ 먼저 이것부터

```
net-stats -l
```

출력이 이렇다.

PortNum	Type	SubType	SwitchName	MACAddress	ClientName
67108942	5	9	DvsPortset-0	00:50:56:xx:xx:xx	web01.eth0
67108988	5	9	DvsPortset-0	00:50:56:yy:yy:yy	web02.eth0

맨 왼쪽 PortNum 이 뒤에 계속 필요하다. 맨 오른쪽에서 VM 이름을 찾아 그 줄의 PortNum 을 쓴다.

E-2 → state/41-esxi-uplink-<nic>-stats.txt

물리 NIC 카운터

```
esxcli network nic stats get -n vmnic0
esxcli network nic stats get -n vmnic1
```

설정의 UPLINK_NICS 에 적은 NIC 마다. 물리 구간 드롭·에러를 본다.

E-3 → state/42-esxi-vmport-<VM>-<port>-stats.txt

VM 포트별 상세 카운터

```
net-stats -A -t vW -p 67108942 -i 1 -n 1
```

옵션	뜻
-p	E-1 에서 찾은 PortNum
-A	모든 통계
-t vW	vNIC · 월드(world) 단위
-i 1 -n 1	1초 간격으로 1회 — 안 주면 계속 돈다

E-4. 반복해서 뜨려면

```
D=/vmfs/volumes/<데이터스토어>/manual-$(date +%Y%m%d-%H%M%S); mkdir -p $D/state
PORT=67108942

END=$(( $(date +%s) + 1200 ))
while [ $(date +%s) -lt $END ]; do
    TS=$(date +%H%M%S)
    net-stats -l >> $D/state/40-switchports.txt
    esxcli network nic stats get -n vmnic0 > $D/state/41-vmnic0-$TS.txt
    net-stats -A -t vW -p $PORT -i 1 -n 1 > $D/state/42-port$PORT-$TS.txt
    sleep 30
done
```

ESXi — 세션 수집 (DFW)

도구의 `dfw-run` · 플로우 테이블 · 적용된 룰 · pass/drop 카운터.

F-0. 먼저 DFW 필터 이름을 찾는다 — 모든 명령의 출발점

```
summarize-dvfilter
```

출력에서 VM 을 찾는다.

```
world 2098765 vmm0:web01 vcUuid:'...'
port 67108942 web01.eth0
vNic slot 2
  name: nic-2098765-eth0-vmware-sfw.2    # ← 이것이 <filter>
  agentName: vmware-sfw
```

읽는 법

- world ... vmm0:<VM이름> — 이 줄로 VM 을 찾는다
- name: 줄의 ...vmware-sfw.N — 이것이 DFW 필터 이름
- vNIC 이 여러 개면 필터도 여러 개다. 전부 떼야 한다

F-1 → session/60-dfw-filter-list.txt

필터 목록 전체를 보관

```
summarize-dvfilter > $D/session/60-dfw-filter-list.txt
```

나중에 "그때 이 VM 이 어느 포트·어느 필터였나"를 대조하려면 원본을 남겨야 한다.

F-2 → session/61-dfw-flows-<VM>-<nic>-...txt

DFW 플로우 테이블 ★

```
vsipioctl getflows -f <filter>
```

이 vNIC 을 지나는 연결이 지금 몇 개고 어떤 것인지. Edge 의 `get firewall ... connection` 에 해당하는 ESXi 쪽 명령이다.

사후에 얻을 수 없다. 증상 진행 중에 떼야 한다.

F-3 → session/62-dfw-rules-<VM>-<nic>.txt

이 vNIC 에 실제로 적용된 룰

```
vsipioctl getrules -f <filter>
```

UI 에 보이는 정책이 아니라 커널에 실제로 내려간 룰이다. "정책은 만들었는데 왜 안 먹나"를 볼 때 여기서 확인한다.

F-4 → session/63-dfw-passdrop-<VM>-<nic>.txt

pass / drop 카운터 ★

```
vsipioctl getfilterstat -f <filter>
```

출력에서 v4 pass · v4 drop 줄을 본다.

가장 빠른 판정 도구

두 시점에 떠서 drop 이 늘었는지만 보면 "DFW 가 막고 있나"가 바로 나온다. 늘지 않았으면 DFW 가 아니라 다른 구간을 봐야 한다.

F-5. 한 번에 붙여 쓰기

```
D=/vmfs/volumes/<데이터스토어>/manual-$(date +%Y%m%d-%H%M%S); mkdir -p $D/session
F=nic-2098765-eth0-vmware-sfw.2 # F-0 에서 찾은 필터
```

```
summarize-dvfilter > $D/session/60-filter-list.txt
```

```
END=$(( $(date +%s) + 1200 ))
while [ $(date +%s) -lt $END ]; do
    TS=$(date +%H%M%S)
    vsipioctl getflows -f $F > $D/session/61-flows-$TS.txt
    vsipioctl getrules -f $F > $D/session/62-rules-$TS.txt
    vsipioctl getfilterstat -f $F > $D/session/63-passdrop-$TS.txt
    sleep 30
done
```

ESXi — 패킷 덤프

도구의 `cap-pre` · `cap-post` . DFW 적용 전/후를 같은 vNIC 에서 뜬다.

원리 — 전과 후를 같이 떼서 대조한다

vNIC → [pre] → DFW 를 평가 → [post] → 스위치

pre 에는 있는데 post 에 없다 = **DFW 가 막았다**

이것이 ESXi 캡처의 핵심이다. **추정이 아니라 증거**가 된다.

먼저 알아야 할 것 — `pktcap-uw` 에는 BPF 필터가 없다

`tcpdump` 처럼 'host A and port B' 같은 표현식을 못 쓴다. `--ip` , `--tcpport` 같은 개별 옵션만 있고 `or` 조합이 안 된다.

그래서 방법이 둘로 갈린다 — **G-1**(옵션만) 또는 **G-2**(`tcpdump-uw` 로 파이프, 권장).

G-0. ESXi 의 timeout 문법부터 확인

ESXi 의 `timeout` 은 버전에 따라 문법이 다르다. **둘 중 되는 것을 쓴다.**

```
timeout -t 1 -s INT true    && echo "형식 A: timeout -t <초> -s INT"
timeout -s INT 1 true       && echo "형식 B: timeout -s INT <초>"
```

`pktcap-uw` 의 -G 는 캡처 시간이 아니다

`-G` 는 **파일 회전 주기**다. 캡처를 시간으로 끝내려면 반드시 **바깥에서** `timeout` 으로 감싸야 한다.

G-1 · 옵션만 쓰는 방식

단순한 조건일 때

```
echo Y | timeout -t 300 -s INT \  
  pktcap-uw --dvfilter <filter> --stage pre \  
  --ip 10.1.1.10 --tcpport 80 \  
  -C 100 -W 5 \  
  -o $D/pcap/20-dfw-pre-web01-eth0.pcap
```

옵션	뜻
<code>--dvfilter</code>	F-0 에서 찾은 DFW 필터 이름
<code>--stage pre post</code>	DFW 적용 전 / 후. 두 번 떼서 대조한다
<code>--ip</code> · <code>--tcpport</code>	필터 옵션. or 조합 불가
<code>-C 100 -W 5</code>	파일당 MB · 개수
<code>echo Y </code>	확인 프롬프트를 자동으로 넘긴다

조건이 여러 개면 파일이 폭증한다

`or` 가 없으니 **조합마다 별도 캡처**를 돌려야 한다. IP 2개 × 포트 2개면 vNIC 당 4개. vNIC 2개면 8개 파일이 된다.

G-2 · tcpdump-uw 로 파이프 권장

복잡한 필터를 쓰고 파일을 하나로

```
echo Y | timeout -t 300 -s INT \  
  pktcap-uw --dvfilter <filter> --stage pre -o - 2>/dev/null \  
  | tcpdump-uw -r - -w $D/pcap/20-dfw-pre-web01-eth0.pcap \  
  -C 100 -W 5 'host 10.1.1.10 and tcp port 80'
```

이 방식의 장점

- **tcpdump 표현식을 그대로 쓴다** — `or`, `not`, 괄호 전부 가능
- **vNIC·단계당 파일 하나** — 파일 폭증이 없다
- 도구가 **FILTER** 설정이 있을 때 쓰는 방식이 이것이다

`pktcap-uw -o -` 로 표준출력에 흘리고, `tcpdump-uw -r -` 로 받아서 필터링·저장한다.

G-3. pre 와 post 를 같이 — 실제로 쓰는 형태

```
D=/vmfs/volumes/<데이터스토어>/manual-$(date +%Y%m%d-%H%M%S); mkdir -p $D/pcap
F=nic-2098765-eth0-vmware-sfw.2
EXPR='host 10.1.1.10 and tcp port 80'

for S in pre post; do
  N=20; [ $S = post ] && N=21
  echo Y | timeout -t 300 -s INT \
    pktcap-uw --dvfilter $F --stage $S -o - 2>/dev/null \
    | tcpdump-uw -r - -w $D/pcap/$N-dfw-$S-web01-eth0.pcap -C 100 -W 5 "$EXPR" &
done
wait
ls -la $D/pcap/
```

반드시 동시에 떼야 한다

pre 와 post 를 **따로 뜨면 대조가 안 된다**. 같은 패킷을 봐야 하므로 & 로 동시에 띄우고 wait 로 기다린다.

G-4. 캡처가 안 될 때

증상	확인
바로 끝나고 파일이 0바이트	2>/dev/null 을 빼고 다시 돌려 에러를 본다. 필터 이름 오타가 가장 흔하다
프롬프트에서 멈춤	echo Y 를 빠뜨렸다
끝나지 않는다	timeout 문법이 이 장비와 안 맞는다 → G-0
패킷이 0건	필터가 안 맞거나, 그 시간에 트래픽이 없었다
디스크가 찼다	/tmp 에 쓰고 있다 → 데이터스토어로

한 장 요약

출력해서 옆에 두고 쓰는 용도.

NSX Edge

구분	명령
상태	su admin -c "get interfaces"
	su admin -c "get dataplane cpu stats"
	su admin -c "get interface fp-eth0" (fp-eth1 도)
	su admin -c "get logical-router <SR> interfaces stats"
	su admin -c "get logical-router <SR> high-availability status" ★
	su admin -c "get cpu-stats" · su admin -c "get memory"
세션	su admin -c "get firewall <LIF> connection count"
	su admin -c "get firewall <LIF> connection" ★
	su admin -c "get load-balancer <LB> status" · stats
	su admin -c "get load-balancer <LB> virtual-servers"
	su admin -c "get load-balancer <LB> pool <POOL> status"
패킷	su admin -c "set capture session 0 interface <LIF> direction dual"
	timeout -s INT 300 tcpdump -nei span-0 -Z root -C 100 -W 5 -w <파일> '<식>'
	su admin -c "del capture session 0" ← 반드시

ESXi

구분	명령
상태	<code>net-stats -l</code> ★ (PortNum 을 여기서 찾는다)
	<code>esxcli network nic stats get -n vmnic0</code>
	<code>net-stats -A -t vW -p <PortNum> -i 1 -n 1</code>
세션	<code>summarize-dvfilter</code> ★ (필터 이름을 여기서 찾는다)
	<code>vsipioctl getflows -f <filter></code> ★
	<code>vsipioctl getrules -f <filter></code>
	<code>vsipioctl getfilterstat -f <filter></code> ★
패킷	<code>echo Y timeout -t 300 -s INT pktcap-uw --dvfilter <f> --stage pre --ip <IP> -o <파일></code>
	<code>echo Y timeout -t 300 -s INT pktcap-uw --dvfilter <f> --stage pre -o - tcpdump-uw -r - -w <파일> '<식>'</code>

실측으로 확인된 함정 6가지

함정	내용
<code>get interfaces stats</code>	그런 명령은 없다(NSX 4.2.4). <code>get interface <이름></code> 을 쓴다
<code>get system-stats</code>	없다. <code>get cpu-stats</code> + <code>get memory</code> 로 나눈다
<code>get firewall <SR> connection</code>	SR 이 아니라 LIF UUID 를 넣어야 한다. SR 을 넣으면 에러가 파일에 담긴다
HA 상태의 <code>Peer Routers</code>	맨 위 <code>state</code> 가 이 노드, Peer 블록은 상대방
<code>tcpdump -Z root</code>	빠뜨리면 링 버퍼 두 번째 파일부터 못 쓴다
<code>pktcap-uw -G</code>	캡처 시간이 아니라 파일 회전 주기. 종료는 <code>timeout</code> 으로

손으로 할 때 가장 많이 잊는 것

1. **span 미러를 안 지운다** — 운영 장비에 계속 부하가 남는다
2. **한 번만 뜯다** — 카운터·세션은 **두 시점 이상** 있어야 의미가 있다
3. **Standby Edge 에서 뜯다** — 0건이 나온다
4. **ESXi /tmp 에 쓴다** — 250MB 램디스크라 금방 찬다

nsx-collector 별첨 — 커맨드 레벨 개별 수집 v1.0 · 2026-09-21 · 도구 v6.3

본편: nsx-collector 사용법 v2.0 · 모든 명령은 `nsx-collector.py` 소스에서 추출했다.